



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	BALAJIAH GUDIPUDI
Student Name	PALAVALASA RAHUL
Roll Number	2520090016
Branch	CSIT
Course Outcome	COURSE OUTCOME - 4

Case Study Topic: BMTC Bus Route Optimization using Bellman-Ford Algorithm

Work Done Summary

In this case study, we analyzed the BMTC bus-route network consisting of seven major hubs in Bangalore. Since one route (Whitefield → Marathahalli) has a negative weight due to a dedicated bus-lane bonus, Dijkstra's algorithm cannot guarantee correct shortest paths. Therefore, Bellman-Ford was used to compute shortest travel times from Majestic (MJC) to all other hubs.

The implementation demonstrates shortest-path computation, convergence analysis, negative-cycle detection, and comparison with Floyd-Warshall for large-scale routing queries.

Implementation Details

1. Represented BMTC hubs as graph vertices.
2. Stored route travel times as directed weighted edges.
3. Applied Dijkstra by temporarily replacing negative edge (-3) with 0.
4. Applied Bellman-Ford on actual graph.
5. Tracked distance array after every iteration.
6. Added negative-cycle detection.
7. Compared Bellman-Ford with Floyd-Warshall.

(a) Dijkstra Result (Treating -3 as 0)

Assumed Edges

MJC → KEM = 8
MJC → JAY = 5
KEM → KOR = 12
KEM → WHF = 7
JAY → KOR = 10
JAY → HBR = 4
KOR → WHF = 9
KOR → MRT = 6
WHF → MRT = 0
HBR → WHF = 3
HBR → MRT = 11

Shortest Distance Table

Hub Distance from MJC

MJC 0

KEM 8

Hub Distance from MJC

JAY 5

KOR 15

HBR 9

WHF 12

MRT 15

Incorrect Destination

When actual edge weight:

$$\text{WHF} \rightarrow \text{MRT} = -3$$

then

$$\text{MJC} \rightarrow \text{JAY} \rightarrow \text{HBR} \rightarrow \text{WHF} \rightarrow \text{MRT}$$

$$= 5 + 4 + 3 + (-3)$$

$$= 9$$

Actual shortest distance:

$$\text{MRT} = 9$$

Dijkstra computes:

$$\text{MRT} = 15$$

Therefore:

Incorrect Vertex

Marathahalli (MRT)

Why Dijkstra Fails

Dijkstra permanently finalizes a vertex once it appears to have the smallest distance. With negative edges, a shorter path may be discovered later through another route. Therefore the greedy decision becomes unsafe and the algorithm may return incorrect shortest paths.

(b) Bellman-Ford Iterations

Actual Edge:

$$\text{WHF} \rightarrow \text{MRT} = -3$$

Initial:

Iteration	MJC	KEM	JAY	KOR	WHF	HBR	MRT
-----------	-----	-----	-----	-----	-----	-----	-----

0	0	∞	∞	∞	∞	∞	∞
---	---	----------	----------	----------	----------	----------	----------

Iteration 1

$$\text{MJC} \rightarrow \text{KEM} = 8$$

$$\text{MJC} \rightarrow \text{JAY} = 5$$

$$\text{JAY} \rightarrow \text{HBR} = 9$$

$$\text{HBR} \rightarrow \text{WHF} = 12$$

WHF→MRT = 9
JAY→KOR = 15

Iter MJC KEM JAY KOR WHF HBR MRT

1 0 8 5 15 12 9 9

Iteration 2

No further relaxation.

Iter MJC KEM JAY KOR WHF HBR MRT

2 0 8 5 15 12 9 9

Iteration 3

No change.

Iter MJC KEM JAY KOR WHF HBR MRT

3 0 8 5 15 12 9 9

Iteration 4

No change.

Iter MJC KEM JAY KOR WHF HBR MRT

4 0 8 5 15 12 9 9

Iteration 5

No change.

Iter MJC KEM JAY KOR WHF HBR MRT

5 0 8 5 15 12 9 9

Iteration 6

No change.

Iter MJC KEM JAY KOR WHF HBR MRT

6 0 8 5 15 12 9 9

Convergence

Bellman-Ford converges after Iteration 1.

This signifies:

All shortest paths have already been found.
Further relaxations do not improve any distance.
No negative cycle exists.

(c) Java Program (Set1CO4.java) Output:

C:\Users\DELL\.jdk\openjdk-25.0.2

Shortest Distances:

0 8 5 15 12 9 9

Process finished with exit code 0

Time Complexity

Bellman-Ford:

$$O(V \times E)$$

For:

$$V = 7$$

$$E = 11$$

Operations:

$$7 \times 11 = 77$$

Dijkstra (Binary Heap)

$$O(E \log V)$$

Approximation:

$$11 \times \log_2(7)$$

$$\approx 11 \times 2.8$$

$$\approx 31 \text{ operations}$$

Comparison

Algorithm	Complexity	Approx Ops
-----------	------------	------------

Bellman-Ford	$O(VE)$	77
--------------	---------	----

Dijkstra	$O(E \log V)$	31
----------	---------------	----

Dijkstra is roughly:

$$77 / 31 \approx 2.5 \text{ times faster}$$

when no negative edges exist.

(d) Bellman-Ford vs Floyd-Warshall

Bellman-Ford Approach

For every query:

Run shortest-path computation again

Cost:

$O(V E)$

For one query:

$7 \times 11 = 77 \text{ operations}$

Queries per minute:

600×60

$= 36,000$

Total:

$36,000 \times 77$

$= 2,772,000 \text{ operations}$

Floyd-Warshall Approach

Precompute once:

$O(V^3)$

Operations:

7^3

$= 343$

After precomputation:

Distance lookup = $O(1)$

For 36,000 queries:

$343 + 36,000$

$\approx 36,343 \text{ operations}$

Comparison Table

Approach	Cost per Query	Total Operations (1 min)
Bellman-Ford Every Query	$O(V E)$	2,772,000
Floyd-Warshall Precompute	$O(1)$ lookup	36,343

Result

Bellman-Ford successfully computes shortest paths despite negative edges and confirms the absence of negative cycles. For a production dispatcher serving 36,000 routing requests per minute, Floyd-Warshall is significantly more efficient because it precomputes all-pairs shortest paths once and answers each query in constant time.

Time Complexity Summary

Algorithm	Complexity
Dijkstra	$O(E \log V)$
Bellman-Ford	$O(V E)$

Algorithm Complexity

Floyd-Warshall $O(V^3)$

Conclusion

The presence of the negative edge (WHF $\rightarrow$ MRT = -3) makes Dijkstra unsafe. Bellman-Ford correctly computes shortest routes and detects negative cycles if they exist. However, for high-frequency dispatch systems processing thousands of queries per second, Floyd-Warshall is the preferred production solution because all shortest paths are precomputed and retrieved instantly.

GitHub Link: <https://github.com/palavalasarahul-blip/DSA-CASE-STUDIES.git>

Student Signature	Faculty Signature
-------------------	-------------------